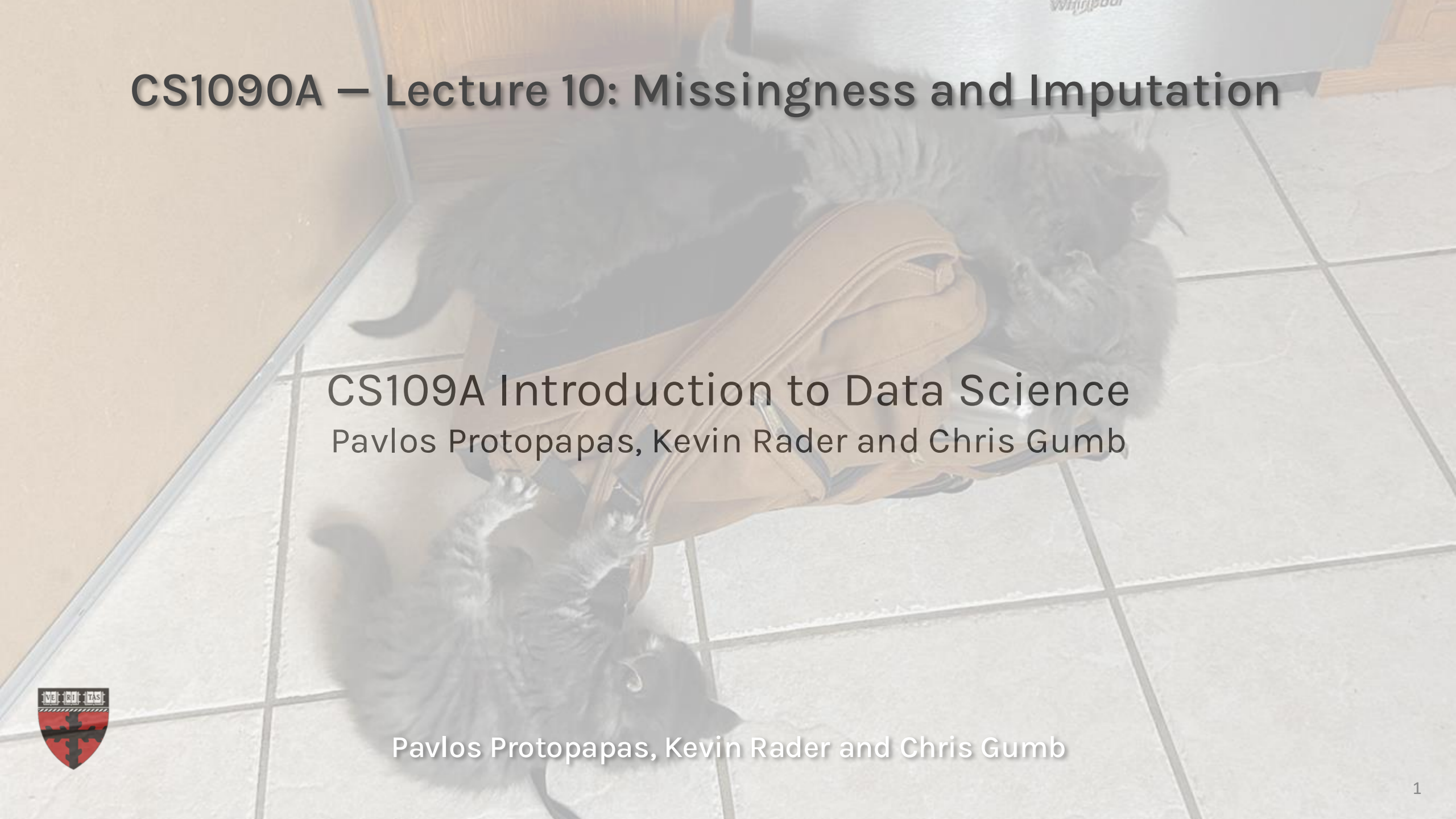
CS1090A — Lecture 10: Missingness and Imputation



CS109A Introduction to Data Science

Pavlos Protopapas, Kevin Rader and Chris Gumb

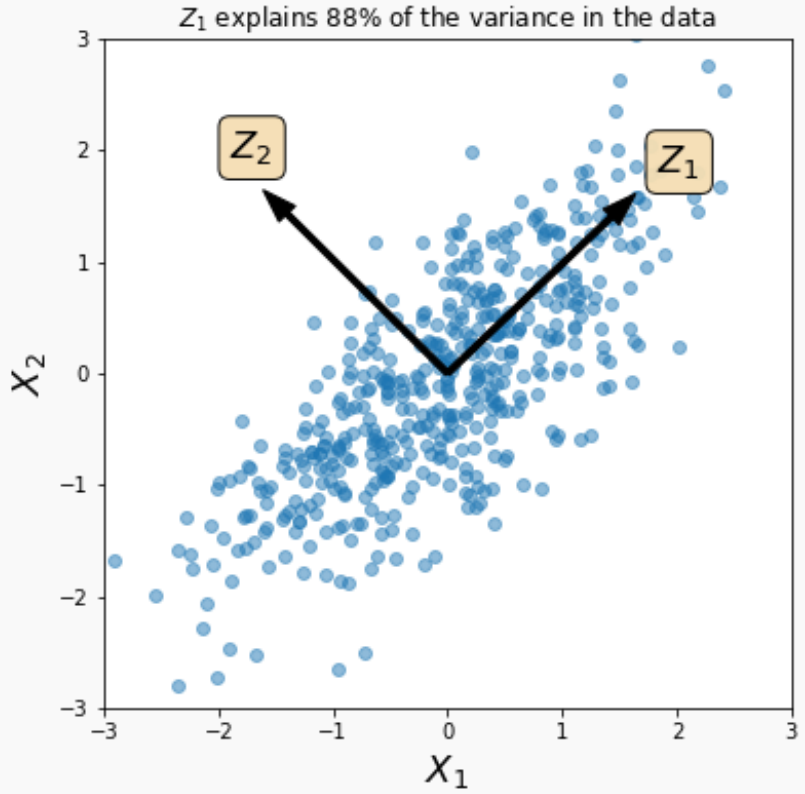
Pavlos Protopapas, Kevin Rader and Chris Gumb

Lecture Outline

- PCA Recap
- Dealing with Missingness
  - Types of Missingness
- Imputation Methods

Directions of maximum variance

The vector Z_1 represents the direction of maximum variance in the predictor space. Z_2 is orthogonal and captures the remaining unexplained variance.



If our data were represented in terms of Z_1 and Z_2 then we could decide to keep only the Z_1 component.

This would again cut the dimensionality in half while retaining the most information.

The Math behind PCA (for those whose are curious)

Let $\boldsymbol{X}$ be the n x p matrix with columns X_1,..., X_p (our original predictors), each standardized to have mean zero and variance one, and without the intercept)

Let let $\boldsymbol{W}$ be the p x p matrix whose columns are the eigenvectors of the covariance matrix, $\boldsymbol{X}^T\boldsymbol{X}$

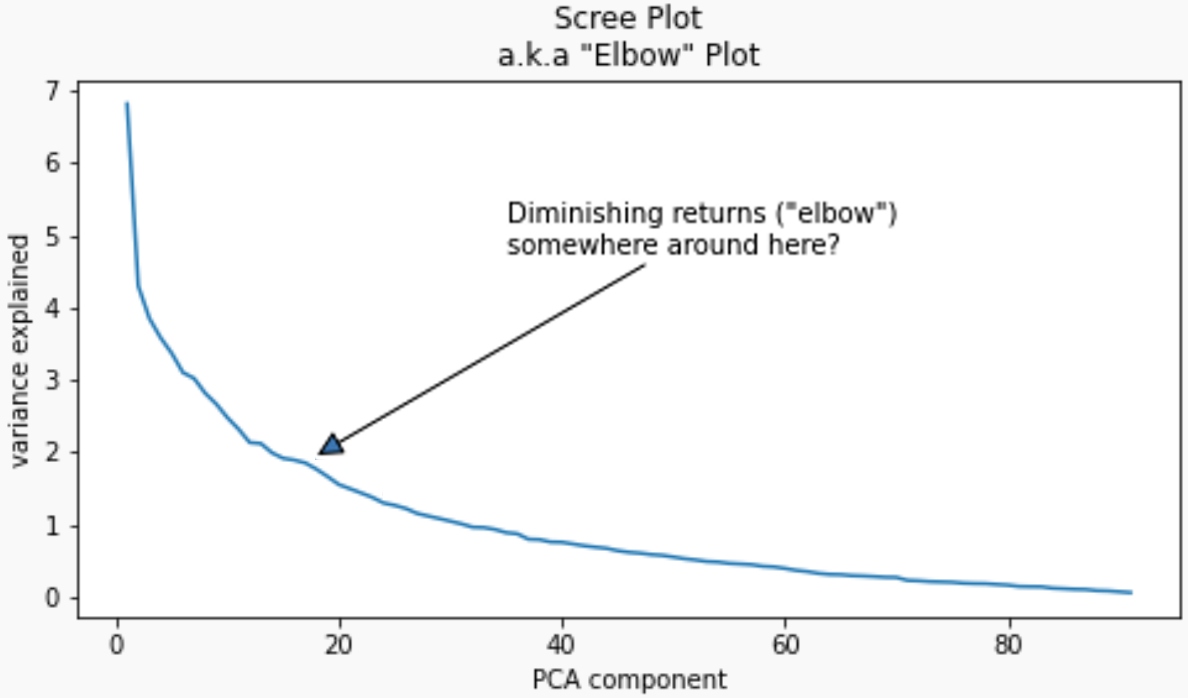
Let $\boldsymbol{Z}$ be the n x p matrix with columns Z_1,..., Z_p (the full set of principal components):

$$\mathbf{Z}_{n\times p} = \mathbf{X}_{n\times p}\mathbf{W}_{p\times p}$$

How many components to keep?

One approach for deciding on the number of components to keep is to just “eyeball” it.

We look for a point of diminishing returns, or “elbow”, in a plot showing the variance explained by each component.





Diminishing returns (“elbow”) somewhere around here?

Interpreting the Results of PCR

A PCR can be interpreted in terms of original predictors…very carefully.

Each estimated β coefficient in the PCR can be *distributed* across the predictors via the associated component vector, *w*.

An example is worth a thousand words:

```
pcaX_df = pd.DataFrame(pcaX, columns=[f'PCA{i}' for i in range(1,X.shape[1]+1)])

PCR_simple = sk.linear_model.LinearRegression().fit(X = pcaX_df[['PCA1']], y = homes['price'])
print("PCA Intercept =",PCR_simple.intercept_.round(2),", PCA Slope =",PCR_simple.coef_[0].round(4))

print("First PCA Component vector (w1):",W[0,:])
```

```
PCA Intercept = 1244.2 , PCA Slope = 0.2049
First PCA Component vector (w1): [ 3.23799939e-01  9.46124307e-01 -7.23671683e-04  9.22493237e-04
  1.00975436e-03]
```

So how can this be transformed back to the original variables?

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 Z_1 = \hat{\beta}_0 + \hat{\beta}_1(\vec{w}_1^T X) = \hat{\beta}_0 + \left(\hat{\beta}_1 \vec{w}_1^T\right) X$$

$$= 1244.2 + 0.2049 \cdot (0.3238X_1 + 0.9461X_2 - 0.00072\, X_3 + 0.00092X_4 + 0.00101X_5)$$

$$= 1244.2 + 0.0663X_1 + 0.194X_2 - 0.00015\, X_3 + 0.00019X_4 + 0.00021X_5$$

Interpreting the Results of PCR

You can always put the PCR coefficients in terms of the original predictors.

$$\hat{y} = \boldsymbol{Z}\beta_Z = (\boldsymbol{X}\boldsymbol{W}_m)\beta_Z = \boldsymbol{X}(\boldsymbol{W}_m\beta_Z) = \boldsymbol{X}\beta_X$$

$\boldsymbol{X}$ is our original dataset, centered to have a mean of zero
$\boldsymbol{W}_m$ is the matrix whose columns are the first m eigenvectors of $\boldsymbol{X}^\top\boldsymbol{X}$
$\boldsymbol{Z}_m$ is the transformed dataset created by projecting $\boldsymbol{X}$ onto the principal components contained in $\boldsymbol{W}_m$
β_Z are the coefficients for the PCR model
β_X are coefficients in terms of the original predictors

When $m = p$ then the recovered coefficients are identical to those from a model fit on the original predictors.

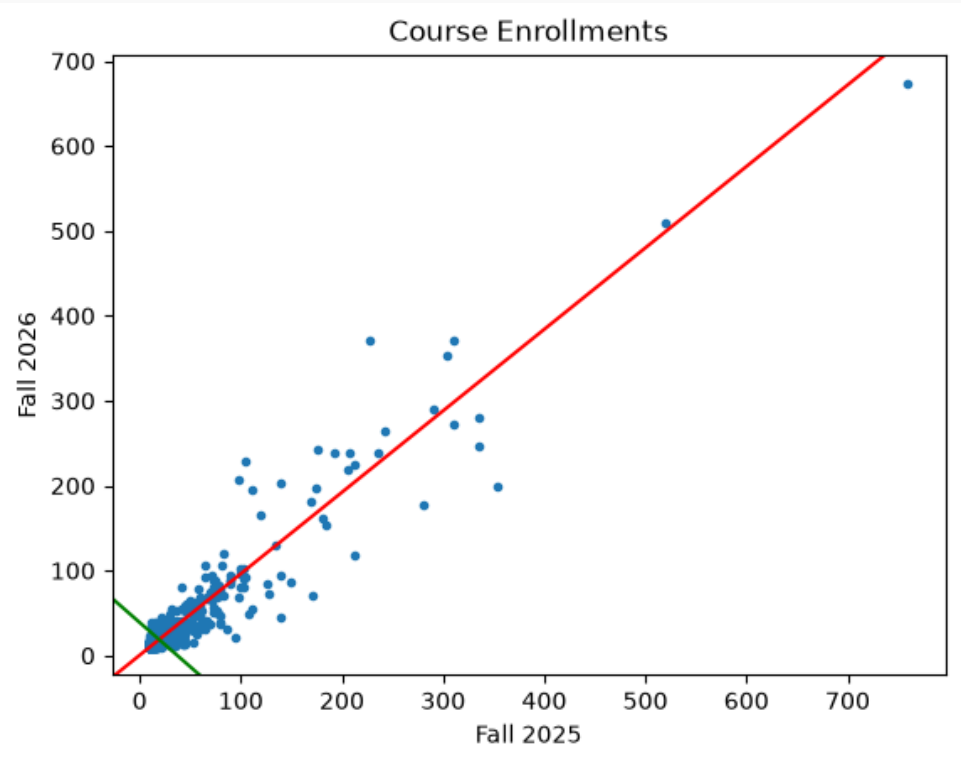
A few notes on using PCA

- PCA cons:
    1. PCA *knows nothing about the response variable*. Component vectors as predictors might not be ordered from best to worst!
    2. *Direct* interpretation of coefficients in PCR is lost, so if easy interpretation is important to you, perhaps steer clear.
    3. PCA can often fail to improve the predictive power of a model.
- PCA pros:
    1. Can help avoid the curse of dimensionality and overfitting.
    2. Aids in visualizing how potentially predictive your features may be for the response variable, especially in the classification setting.
    3. Reduces multicollinearity, and so can improve the computational time when fitting models.

An *intuitive* PCA example

```
classes = pd.read_csv('data/classes.csv')
classes.head()
```

	Course.Title	Total25	Total26
0	AFRAMER 134Y	23	20
1	AFRAMER 392	15	13
2	AFVS 390	10	11
3	AFVS 53AR	12	11
4	AFVS 99A	19	14



What should be Z_1 and Z_2? What do they represent?

```
X = classes[["Total25","Total26"]]
pca = PCA().fit(X)
W = pca.components_
W
```

```
array([[ 0.72166011,  0.69224756],
       [-0.69224756,  0.72166011]])
```

```
pcaZ = pca.transform(X)
pd.DataFrame(pcaZ).head()
```

	0	1
0	3.545364	-1.749617
1	-7.073650	-1.263258
2	-12.066446	0.754660
3	-10.623125	-0.629835
4	-3.494762	-3.310588

Note: why does standardizing not really matter here? And why might we choose not to do it?

```
classes[classes["Course.Title"]=="COMPSCI 1090A"]
```

	Course.Title	Total25	Total26
94	COMPSCI 1090A	127	73

Lecture Outline

- PCA Recap
- **Dealing with Missingness**
  - Types of Missingness
- Imputation Methods

What is missing data?

Often times when data is collected, there are some missing values apparent in the dataset. This leads to a few questions to consider:

- How does this show up in pandas?
- How does sklearn handle these NaNs?
- How does this effect our modeling?

What are the simplest ways to handle missing data?

Naively handling missingness

- Drop the observations that have any missing values.
  - Use pd.DataFrame.dropna(axis=0)
- Impute the mean/median (if quantitative) or most common class (if categorical) for all missing values.
  - Use pd.DataFrame.fillna(value=x.mean())

How do statsmodels and sklearn handle these NaNs?

What are some consequences in handling missingness in these fashions?

Missingness Indicator Variable

One simple way to handle missingness in a variable, X_j, is to impute a value (like 0 or $\overline{X_j}$), then create a new variable, $X_{j,miss}$, that indicates this observation had a missing value. If X_j is categorical then just impute 0.

Then include both $X_{j,miss}$ and X_j as predictors in any model.

Illustration is to the right.

X_1	X_2	X_1^*	X_2^*	$X_{1,miss}$	$X_{2,miss}$
10	.	10	0	0	1
5	1	5	1	0	0
21	0	21	0	0	0
15	0	15	0	0	0
16	.	16	0	0	1
.	.	0	0	1	1
21	1	21	1	0	0
12	0	12	0	0	0
.	1	0	1	1	0

Why use a Missingness Indicator Variable?

How does this missingness indicator variable improve the model?

Because the group of individuals with a missing entry may be systematically different than those with that variable measured. Treating them equivalently could lead to bias in quantifying relationships (the β's) and underperform in prediction.

For example: imagine a survey that asks whether or not someone has ever recreationally used opioids, and some people chose not to respond. Does the fact that they did not respond provide extra information? Should we treat them equivalently as never-users?

This approach essentially creates a third group for this predictor: the "did not respond" group.

Lecture Outline

- PCA Recap
- Dealing with Missingness
  - **Types of Missingness**
- Imputation Methods

Sources of Missingness

Missing data can arise from various places in data:

- A survey was conducted and values were just randomly missed when being entered in the computer.
- A respondent chooses not to respond to a question like `Have you ever recreationally used opioids?'.
- You decide to start collecting a new variable (due to new actions: like a pandemic) partway through the data collection of a study.
- You want to measure the speed of meteors, and some observations are just 'too quick' to be measured properly.

The source of missing values in data can lead to the major types of missingness:

Types of Missingness

There are 3 major types of missingness to be concerned about:

1. **Missing Completely at Random (MCAR)** - the probability of missingness in a variable is the same for all units. Like randomly poking holes in a data set.
2. **Missing at Random (MAR)** - the probability of missingness in a variable depends only on available information (in other predictors).
3. **Missing Not at Random (MNAR)** - the probability of missingness depends on information that has not been recorded and this information also predicts the missing values.

What are examples of each these 3 types?

Missing completely at random (MCAR)

Missing Completely at Random is the best case scenario, and the easiest to handle:

- Examples: a coin is flipped to determine whether an entry is removed. Or when values were just randomly missed when being entered in the computer.
- Effect if you ignore: there is no effect on inferences (estimates of β).
- How to handle: lots of options, but best to impute (more on next slide).

Missing at random (MAR)

Missing at Random is still a case that can be handled.

- Example(s): men and women respond to the question "have you ever felt harassed at work?" at different rates (and may be harassed at different rates).
- Effect if you ignore: inferences are biased (estimates of β's) and predictions are usually worsened.
- How to handle: use the information in the other predictors to build a model and **impute** a value for the missing entry.

Key: we can fix any biases by modeling and imputing the missing values based on what is observed!

Missing Not at Random (MNAR)

Missing Not at Random is the worst-case scenario, and impossible to handle properly:

- Example(s): patients drop out of a study because they experience some really bad side effect that was not measured. Or cheaters are less likely to respond when asked if you've ever cheated.
- Effect if you ignore: there are major effects on inferences (estimates of beta) or predictions.
- How to handle: you can 'improve' things by dealing with it like it is MAR, but you [likely] may never completely fix the bias. And incorporating a **missingness indicator variable** may actually be the best approach (if it is in a predictor).

What type of missingness is present?

Can you ever tell based on your data what type of missingness is actually present?

Since we asked the question, the answer must be no.

It generally cannot be determined whether data really are missing at random, or whether the missingness depends on unobserved predictors or the missing data themselves. The problem is that these potential “lurking variables” are unobserved (by definition) and so can never be completely ruled out.

In practice, a model with as many predictors as possible is used so that the `missing at random' assumption is reasonable.

Lecture Outline

- PCA Recap
- Dealing with Missingness
  - Types of Missingness
- **Imputation Methods**

Handling Missing Data

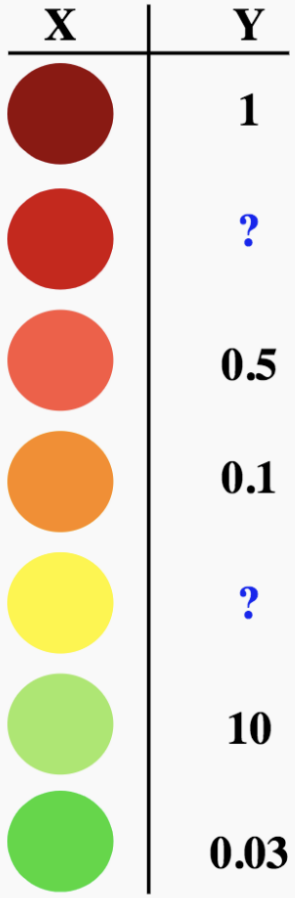
When encountering missing data, the approach to handling it depends on…

1. …whether the missing values are in the response or in the predictors. Generally speaking, it is much easier to handle missingness in predictors.
2. …whether the variable is quantitative or categorical.
3. …how much missingness is present in the variable. If there is too much missingness, you may be doing more harm than good.

Generally speaking, it is a good idea to attempt to **impute** (or 'fill in') entries for missing values in a variable (assuming your method of imputation is a good one).

Schematic: imputation through modeling

How do we impute values for the missing data?



Imputation Methods

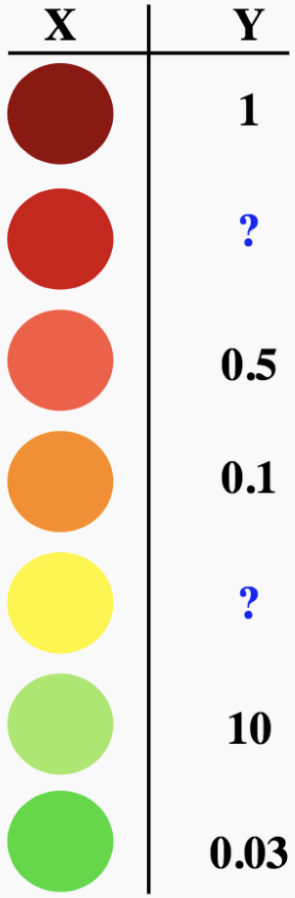
There are several different approaches to imputing missing values:

1. **Impute the mean or median** (quantitative) for all missing values in a variable.
2. Create a new variable that is an **indicator of missingness**, and include it in any model to predict the response (also plug in zero or the mean in the actual variable).
3. **Hot deck imputation**: for each missing entry, randomly select an observed entry in the variable and plug it in.
4. **Model the imputation**: plug in predicted values ($\hat{y}$) from a model based on the other observed predictors.
5. **Model the imputation with uncertainty**: plug in predicted values plus randomness ($\hat{y} + \epsilon$) from a model based on the other observed predictors.

What are the advantages and disadvantages of each approach?

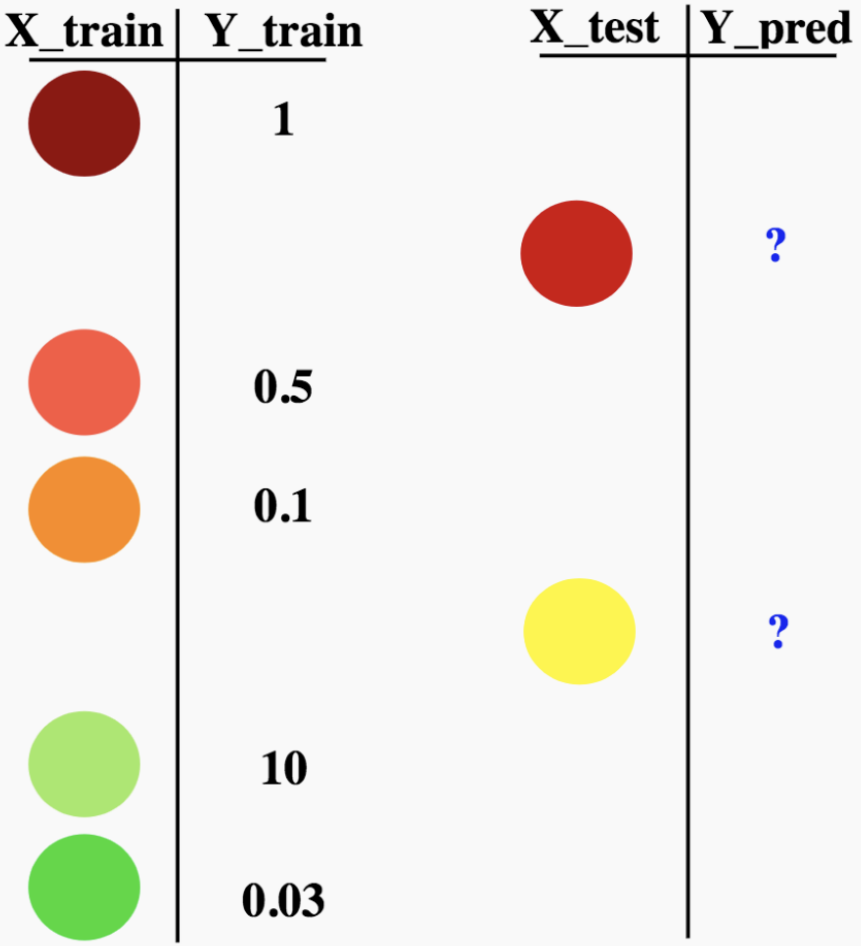
Schematic: imputation through modeling

How do we use models to fill in missing data?



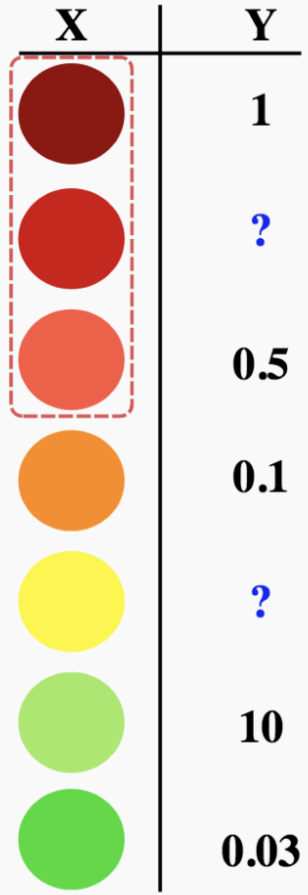
Schematic: imputation through modeling

How do we use models to fill in missing data?



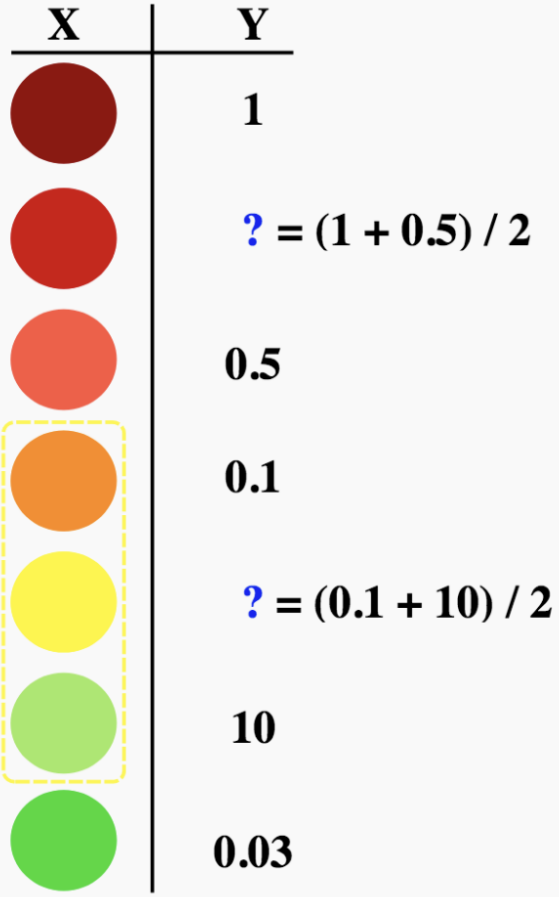
Schematic: imputation through modeling

How do we use models to fill in missing data? Using *k*-NN for *k* = 2?



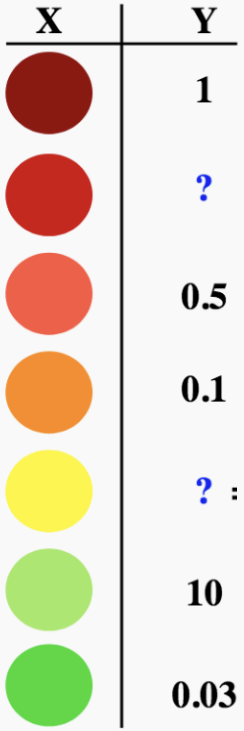
Schematic: imputation through modeling

How do we use models to fill in missing data? Using *k*-NN for *k* = 2?



Schematic: imputation through modeling

How do we use models to fill in missing data? Using linear regression?



Where m and b are computed from the observations (rows) that do not have missingness (we should call them $b = \beta_0$ and $m = \beta_1$).

Imputation through modeling with uncertainty

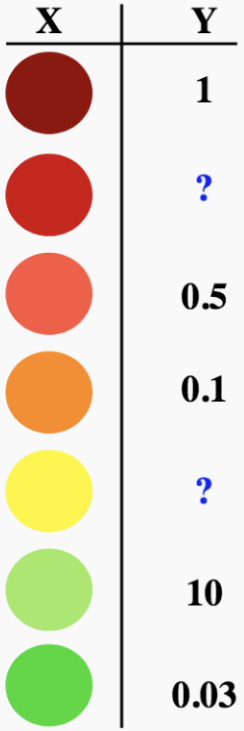
The schematic in the last few slides ignores the fact of imputing with uncertainty. What happens if you ignore this fact and just use the 'best' model to impute values solely on $\hat{y}$?

The distribution of the imputed values will be too narrow and not represent real data (see next slide for illustration). The goal is to impute values that include the uncertainty of the model.

How can this be done in practice in k-NN? In linear regression? In logistic regression?

Schematic: imputation through modeling with uncertainty

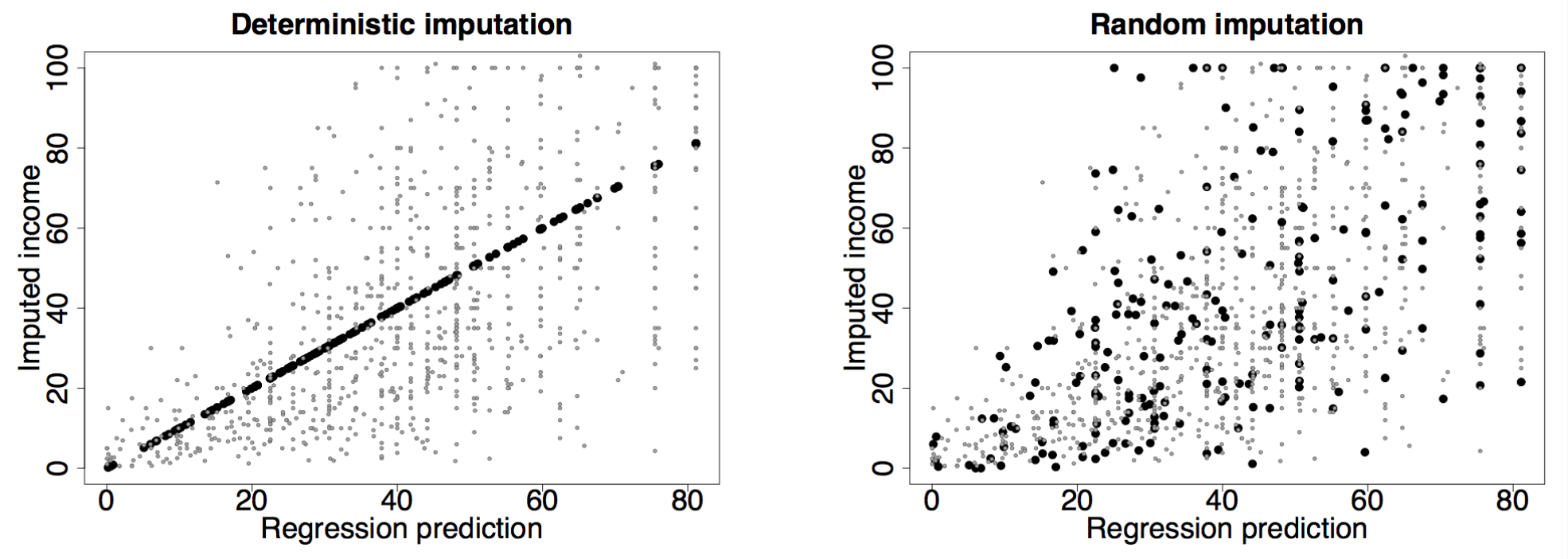
How do we use linear regression to fill in missing data with uncertainty?



How can can sample ε_i and $\varepsilon_{i'}$?

1. Sample from the observed residuals! (sorta like bootstrapping)
2. Sample from an assumed (*estimated*) distribution!

Imputation: modeling with uncertainty (an illustration)



Imputation: modeling with uncertainty (an algorithm)

Recall the probabilistic model in linear regression:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_P + \epsilon \quad \text{where } \epsilon \sim N(0, \sigma^2)$$

How can we take advantage of this model to impute with uncertainty?

It's a 3 step process:

1. Fit a model to predict the predictor variable with missingness from all the other predictors.
2. Predict the missing values from the model in the previous part.
3. Add in a measure of uncertainty to this prediction by randomly sampling from a $N(0, \sigma^2)$ distribution*, where σ^2 is the mean square error (MSE) from the model.

   *Instead of sampling from a $N(0, \sigma^2)$ distribution, you can bootstrap sample from the observed residuals!

Imputation: modeling with uncertainty (k-NN regression)

How can we use k-NN regression to impute values that mimic the error in our observations?

Two ways:

- Use $k = 1$.
- Use any other k, but randomly select from the nearest neighbors in $\mathcal{N}_0$. This can be done with equal probability or with some weighting (inverse to the distance measure used).

Imputation across multiple variables

If only one variable has missing entries, life is easy. But what if all the predictor variables have a little bit of missingness (with some observations having multiple entries missing)? How can we handle that?

It's an iterative process. Impute X_1 based on X_2, ..., X_p. Then impute X_2 based on X_1 and X_3, ..., X_p. And continue down the line.

Any issues? Yes, not all of the missing values may be imputed with just one 'run' through the data set. So you will have to repeat these 'runs' until you have a completely filled in data set.

Multiple imputation: beyond this class

What is an issue with treating your now ‘complete’ data set (a mixture of actually observed values and imputed values) as simply all observed values?

Any inferences or predictions carried out will be tuned and potentially overfit to the random entries imputed for the missing entries. How can we prevent this phenomenon?

By performing **multiple imputation**: rerun the imputation algorithm many times, refit the model on the response many times (one time each), and then ’average’ the predictions or estimates of β coefficients to perform inferences (also incorporating the uncertainty involved).

Note: this is beyond what we would expect in this class. But it generally a good thing to be aware of.

sklearn's impute module

Want to do imputation in sklearn? Of course there are functions for that!

The 4 main functions in this module are:

1. **`sklearn.impute.SimpleImputer`**: perform mean, median, mode, or any specific value to impute
2. **`sklearn.impute.IterativeImputer`**: perform multivariate imputation that estimates each predictor from all the others (user-supplied model/estimator).
3. **`sklearn.impute.KNNImputer`**: perform imputation automatically from a *k*-NN model from all the other predictors.
4. **`sklearn.impute.MissingIndicator`**: to create binary indicator(s) for where the missing values occur.

Imputation through modeling with uncertainty: classifiers

What if the missingness is in a categorical variable? How should we impute value?

1. **Impute the mode**: most common class (categorical)
2. Create a new variable that is an **indicator of missingness**.
3. **Hot deck imputation**: for each missing entry, randomly select an observed entry in the variable and plug it in.
4. **Model the imputation**: plug in predicted values ($\hat{y}$) from a model based on the other observed predictors.
5. **Model the imputation with uncertainty**: plug in predicted values plus randomness ($\hat{y} + \epsilon$) from a model based on the other observed predictors. <- now takes some care

Imputation through modeling with uncertainty: classifiers

For classifiers, this imputation with uncertainty/randomness is a little easier process. How can it be implemented?

If a classification model (logistic, *k*-NN, etc…) is used to predict the variable with missingness on the observed predictors, then all you need to do is flip a 'biased coin' (or multi-sided die) with the probabilities of coming up for each class equal to the predicted probabilities from the model.

Warning: do not just classify blindly using the predict command in sklearn!

What we are trying to avoid…



Insufficient Data.